

构建面向 Agentic 推理的下一代 LLM 推理引擎

—— 从 Runtime 到 Kernel 的系统优化实践

TokenSpeed 研发实践：调度器、自动并行编译、Blackwell MLA Kernel 与跨平台 Kernel 的协同设计

Agentic 负载

Runtime 调度

Kernel 优化

跨平台

580 TPS 实践

演讲人：尚旭春 · TokenSpeed 团队

2026 年 8 月 · AICon 全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon

上海站

全球软件开发大会

大模型正在重新定义软件

Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon

北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

目录 · AGENDA

50 分钟 · 五个部分

01	 为什么 Agentic 推理需要新推理引擎 负载范式转移 · 模型与硬件复杂度同步提升	9 min
02	 Runtime：构建可扩展的推理引擎架构 分层设计 · 自动并行编译 · 请求调度与资源管理	13 min
03	 Kernel：让 MLA 真正跑满 GPU MLA Decode 瓶颈 · Blackwell 优化 · 推测解码 · Prefill	12 min
04	 跨平台 Kernel 设计 Registry + Selector · AMD GPU 实践 (GPT-OSS)	6 min
05	 工程实践与性能收益 Runtime / MLA / 推测解码 / 端到端数据	7 min

01

PART 01

为什么 Agentic 推理需要新推理引擎

Agent 负载的范式转移：从『追求极限吞吐』到『延迟约束下的吞吐』

工作负载新挑战

模型与硬件复杂度

协同设计动机

Agentic 工作负载的新挑战：Agent 与 Chat 的推理模式差异

Chat / 传统对话

-  单轮长对话，生成连续
一次交互通常一段完整生成
-  高 batch 优先
追求极限系统吞吐 (tps/gpu)
-  上下文一次性加载
前缀复用率相对低
-  延迟不敏感
几十 TPS 即可接受

Agentic / 智能体工作负载

-  多轮工具调用，生成碎片化
思考 → 工具 → 思考 → 工具...
-  短请求 × 高并发
每轮生成短，用户侧延迟敏感
-  50K+ 长上下文反复复用
首轮 50K，每轮追加 ~800 token
-  10—15 轮交互
90%+ 前缀可被缓存复用



同一用户请求 = 多段短生成 + 长上下文复用 → 调度器、缓存与 Kernel 的设计优先级被彻底改变

Token Throughput 与 User Latency 的重新平衡



传统引擎目标：极限吞吐

最大化系统吞吐 (tps/gpu)
牺牲单用户延迟换取 batch 效率



Agentic 引擎目标：延迟约束下的吞吐

在保证用户延迟下限 (如 70 TPS/user) 的前提下
最大化每 GPU Token 产出 (tok/min/gpu)



延迟有底线

SLA: 每用户 TPS 下限 70, 部分场景要求
200+



吞吐按 GPU 计

成本视角：相同延迟下每卡产出越多越好



缓存决定上限

前缀命中率直接决定能跑多快

量化效果 (vs TensorRT-LLM, Blackwell, SWE-smith agentic 流量)

bs=1 最小延迟快 ~9%

~100 TPS/user 时每 GPU 吞吐
高 ~11%

KV Cache 成为核心资源



多轮复用

首轮 50K + 10—15 轮追加上下文
Qwen3.5 agentic 实测前缀命中 90%+



长上下文

128K → 1M token 上下文
解码吞吐衰减控制在 ~16%



混合状态

线性注意力 (GDN/KDA) 不只是 KV:
循环状态快照也要可复用 (MambaSlot)



KV Cache 从『临时缓冲』升级为『一等资源』

所有权、生命周期、复用安全全部由调度器统一管理——这是第二部分 Runtime 设计的出发点

前缀缓存

Radix Tree 匹配 + L2/L3 分层存储 + Copy-on-Write

请求级 KV

分页管理、块级引用计数 (RAII)、编译期安全复用

PD 分离传输

KV 与循环状态走同一条 RDMA wire

模型与硬件复杂度同步提升



模型侧：新一代架构成为主流

- ▶ **MoE 稀疏化** Kimi K3: 2.8T 参数 / 896 专家 / 每 token 路由 16
- ▶ **MLA 潜在注意力** 压缩潜在 KV, decode 带宽需求大幅下降
- ▶ **混合注意力** GDN / KDA / 滑动窗口与全注意力交替
- ▶ **4-bit 量化** NVFP4 / MXFP4 成为部署主流格式



硬件侧：新一代 GPU 能力释放

- ▶ **Blackwell (SM100/103)** TMEM · tcgen05 MMA · TMA · warp specialization
- ▶ **CDNA4 / CDNA5 (AMD)** scaled MFMA · XCD 拓扑 · 大共享内存
- ▶ **异构并行成为标配** TP / EP / PD 任意组合, 通信与计算深度耦合
- ▶ **速度差距扩大** 手写/定制 Kernel 与通用实现差距越拉越大



负载 × 架构 × 硬件 三轴同时变化 —— 任何单点优化都无法独撑性能，必须系统化协同

Runtime 与 Kernel 为什么必须协同设计



只优化 Kernel

单算子再快也有天花板：
调度 overhead、缓存未命中、
GPU 空转都会吃掉收益



只优化 Runtime

调度再聪明也无法凭空造出算力：
空转的 Tensor Core 等不到数据



协同设计

调度器决定『何时把什么交给 GPU』
Kernel 决定『GPU 怎么最快跑完』
两者共享同一个目标函数

TokenSpeed 的协同设计



Runtime 拥有调度与生命周期

C++ 控制面：FSM + RAIL，编译期保证资源安全



Kernel 是一等子系统

Registry + Selector：可移植 API + 可插拔高性能实现



边界清晰

Python 执行面只通过公开 Kernel API 触碰硬件

02

PART 02

Runtime：构建可扩展的推理引擎架构

控制面与执行面分离：把『安全』放进类型系统，把『效率』留给编译器

分层架构设计

自动并行编译

请求调度与资源管理

分层架构设计：Control Plane / Execution Plane / Kernel Plane



Control Plane

C++ Scheduler

FSM 生命周期 · KV 所有权 · Overlap 时序 · 前缀匹配

编译期安全

目标：正确性 + 资源安全

nanobind · 事件驱动

↓ ExecutionPlan / Events



Execution Plane

Python ModelExecutor / ModelRunner

图捕获 · 多流并发 · 模型执行 · Kernel 调用

迭代效率

目标：快速迭代 + 低延迟

公开 Kernel API · 多流并发

↓ ExecutionPlan / Events



Kernel Plane

tokenspeed-kernel (Registry + Selector)

MLA / MoE / Attention / 通信 · 多硬件解决方案

可插拔 · 可替换

目标：跨硬件高性能

Triton · CuTe DSL · Gluon · 插件

HARDWARE

NVIDIA Blackwell (SM100/103)

AMD CDNA4 / CDNA5

未来：Vera Rubin / MI455X

自动并行编译 (1/2) : Placement 类型系统

核心思想：用户写『单机单卡语义』的模型代码，并行由编译器生成



PlacementType

REPLICATE (复制)
SHARD (按维切分)
PARTIAL (部分和, 待归约)



ParallelGroup

ATTN_TP · DENSE_TP · MOE_TP_EP
不同模块可挂在不同并行组



ModuleSpec

每个子模块声明 input/output Placement
+ CallConvention + FusionCapability



设计定位

DTensor-inspired 的轻量注解
编译期静态分析, 无运行时开销

模型代码 (概念示意)

```
class MoEBlock(ModuleSpec):
    input_placement = Placement(SHARD, MOE_TP_EP)
    output_placement = Placement(PARTIAL, MOE_TP_EP)
```

```
def forward(x: ShardTensor):
    scores = router(x)    # 单卡语义
    y = moe_gemm(x, experts)
    return weighted_sum(y, scores)
```

编译期自动生成: pre_comms → runner → post_comms

轻量注解, 零运行时开销: 没有 torch.DTensor 的运行时追踪

自动并行编译（2/2）：Local-SPMD Compiler 与自动通信生成



自动生成的通信原语

AllReduce

PARTIAL → REPLICATE

ReduceScatter

PARTIAL → SHARD

AllGather

SHARD → REPLICATE

ResidualAllGather / ResidualSlice

AR ↔ RSAG 模式切换



Deferred Reduce：延迟归约，融合执行

AllReduce 延迟到与下游融合：
FusedReduceNorm 单 kernel 完成
AllReduce + Residual + RMSNorm

3 launch → 1

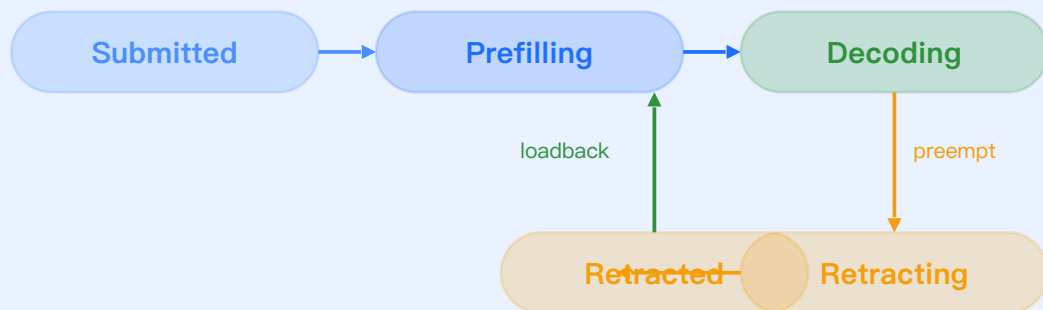
token 数 ≤ 阈值时自动启用
(comm_fusion_max_num_tokens)

收益：模型代码与并行策略解耦 —— 同一份模型，TP/EP 任意组合；通信与归一化融合后每层从 3 次 launch 降为 1 次

来源：python/tokenspeed/runtime/models/base/{placement,module_spec,compiler,comm_ops}.py

请求调度：C++ FSM 生命周期管理与 RAI

请求生命周期 = 显式 FSM



(另有 WritingBack / Draining / Aborting / Prefetching / Finished 等状态)

`std::variant<State>` · 状态即资源容器

事件驱动迁移: `std::visit + operator()`

移动语义转移所有权



状态 = 资源容器

每个状态持有 `unique_ptr` 的 KV 引用、`LocalKVAllocator`、`BlockTable`；状态转移即资源转移



CacheBlockRef (引用计数)

释放最后一个引用才归还块 —— 资源安全由类型系统保证，而非运行时检查



DeviceNodeRef / HostNodeRef

RAII 锁钉住缓存页，直到写回 / 加载完成



check_device_mem 校验

每轮调度校验: radix 树 + 请求尾页 + free 页 严格无重叠

『把资源安全从运行时断言，提前到编译期类型系统』

KV Cache 生命周期: Retraction 与 Prefix Cache



前缀缓存 (HybridPrefixCache)

- ✓ Radix Tree 前缀匹配, KV 与循环状态统一进一棵树
- ✓ Checkpoint 只读复用 + Copy-on-Write, 请求私有 working slot
- ✓ L2 host 缓存 / L3 存储分层, 异步 prefetch / loadback / writeback



Retraction (抢占/换出)

- ✓ 设备页写回 host (Insert + StateRecovery), 请求进入 Retracted
- ✓ 恢复时从 host loadback 重新开始 decode
- ✓ FlatKV 饥饿保护: 连续 wedge 后 retract 最大持有者



PD 分离状态传输

- ✓ KV + 循环状态走同一条 RDMA wire (统一编址)
- ✓ 统一步进计数器: 层完成即传输, 计算通信重叠
- ✓ 三阶段握手: 状态 + 首 token 原子到达, decode 不空转



Kimi K3 案例: 统一扁平 KV 缓存 (FlatKV)

所有缓存类型共用单一物理 page 池: 一个 page 存 1,536 token 的 MLA 潜在历史, 或一个 KDA 循环快照; 全局 page-ID 空间同时支撑前缀缓存、Copy-on-Write 与 CUDA-graph decode

03

PART 03

Kernel: 让 MLA 真正跑满 GPU

从『能用』到『跑满』：解码瓶颈、Blackwell 新原语、推测解码与 Prefill

MLA Decode 瓶颈

Blackwell 优化

Speculative Decoding

Prefill

MLA Decode 的性能瓶颈



MLA：压缩潜在注意力

每 token 的 KV 被压缩为单一潜在状态 (latent KV)，cache 量大幅下降——但 decode 的矩阵形状变『瘦』了

decode 每步：BMM1 (query × latent KV)
BMM2 (attn × V)
query 只有 1 个 token (逐 token 生成)

BMM1 的 M 维度 = 1



Tensor Core 利用率低

M=1 的矩阵乘，BMM1 几乎退化成 GEMV，计算密度不足



带宽受限

长前缀下必须流式读取整个 KV：50K—1M token 的 latent KV，FP8 也不够



头数与序列不匹配

num_heads 与 q_seqlen 组合不佳，矩阵『瘦高』，SIMT 与 Tensor Core 双低效



启动开销放大

短请求多轮调用，每轮多次 kernel launch，延迟被固定开销占满

优化目标：长前缀 + 推测解码场景下，把 MLA decode 延迟压到 TRT-LLM 的一半左右（详见第五部分数据）

来源：lightseek.org/blog/lightseek-tokenspeed.html；tokenspeed-mla/README.md

Blackwell Kernel 优化：让 MLA 吃满 TMEM 与 tcgen05



TMEM 分配器

显式分配/等待/取指针/释放
(sm_100/103: 512 列), tcgen05 MMA 直接读写 TMEM



Warp Specialization

TMA 异步加载 + producer/consumer 流水, 隐藏 KV 读取延迟



fold_sq_factor

num_heads < 128 时 query 折入 head 轴
 $H_{eff} = heads \times F$, 提升 BMM1 M 维利用率



split-KV 两阶段

主 kernel + 归约 kernel; split_kv / workspace 运行时自动计算并缓存



2CTA UTCMMA

减少共享内存占用, mbarrier 数量最小化



带宽友好 epilogue

多级 STG + 最优排布, 避免写回成为新瓶颈



NVFP4 GEMM + SwiGLU + 量化融合: tcgen05.mma · 2CTA · TMEM 内 SFA/SFB/Accumulator 排布 —— 单 kernel 完成 4-bit 专家计算

Speculative Decoding 优化：把『移动数据』变成『移动指针』



多算法支持

EAGLE3 / MTP / DFLASH · draft steps=3 · top-k=1

DeepSeek V4 MTP、Kimi K3 NextN、GLM5 NextN、Inkling MTP 均已落地

draft 模型与 target 同卡执行，single CUDA graph 一次 replay

Move Pointers, Not Data

Mamba/线性注意力状态的推测验证，传统做法是缓存中间状态、验证后整张复制回滚；我们改为：

- 1 每个请求的 draft 步状态直接写入私有物理行（draft region）
- 2 current_input_indices 只记录『当前状态在哪个物理行』
- 3 验证通过后仅更新索引 —— $O(1)$ 整数写，取代 $O(L \times D)$ 整张拷贝

2.08x—2.33x

Inkling 975B · B200x4 · MTP 3/1/4 → 317.5 TPS ·
8/1/9 → 354.6 TPS（关闭 152.4）

+100%—+159%

Qwen3.5-397B bs=1 MTP 收益（长输出 bs=32/64 仍
+38%~+90%）

40K tok/s

Inkling 并发=4 时系统吞吐（MTP 关闭口径）

▀▀ Prefill 优化：长前缀、分块与编译产物



Ragged varlen 无 padding

Q/KV 不同 seq 可同包处理；causal / non-causal 双模式；输出 LSE



双后端：JIT + AOT

CuTe DSL JIT（默认）+ TVM-FFI AOT binary（调优 softmax、PDL、skip-correction）



Chunked prefill

调度器按预算发 chunk；K3 chunk 自动对齐 FlatKV 页粒度（32K→1472 / 1M→1536）



图化与热身

PrefillGraph + eager max-chunk warmup；FA4 支持（head_dim=256 已合入上游）

AMD 侧（Kimi K3 Gated MLA prefill）：256 个 persistent workgroups 跨 XCD 均衡调度 · K/V 双缓冲重叠异步加载与 NoPE/RoPE scoring

04

PART 04

跨平台 Kernel 设计

一套 Runtime，多块硬件：把『平台能力』变成可注册的一等公民

Registry + Selector

AMD GPU 实践 (GPT-OSS)

Registry + Selector: 统一 API 如何支持多种硬件



公开 API (平台无关 · 实现无关)

mha_prefill / mha_decode_with_kvcache / moe_apply ...



select_kernel

按 capability / signature / traits 过滤 → 排序 → 缓存结果



Kernel Registry

@register_kernel(family, mode, solution, capability, traits, priority)



Solutions

Triton (可移植) · CuTe DSL (NVIDIA) · Gluon (AMD) · 厂商 wrapper (TRT-LLM / FlashInfer) · 插件

Priority 分层

REFERENCE	0
PORTABLE (Triton)	4-7
PERFORMANT	8-11
SPECIALIZED (定制)	12-15
PLUGIN (生态)	16-19

关键特性

- ✓ 模型只描述『问题』，不指名『实现』
- ✓ 支持 override 强制指定，便于调试与 benchmark
- ✓ numerics / benchmark / profiling 复用同一注册元数据
- ✓ tokenspeed-kernel-amd 已被 vLLM 采用 (PR #46742)

AMD GPU 实践：GPT-OSS 在 MI355X (Gluon)



Gluon: Triton 家族 DSL

显式暴露 CDNA4 能力，而非隐藏编译器魔法：

- ▶ scaled MFMA (FP8/MXFP 矩阵核)
- ▶ 异步拷贝 / buffer 加载 / 共享内存布局
- ▶ Swizzled / Padded SharedLayout 防 bank conflict
- ▶ 软件流水线显式化：多缓冲 + `async_wait`

decode 瓶颈是隐藏内存延迟 + 保持矩阵核忙碌 —— 流水线深度由 kernel 作者显式控制

14 / 15

Attention: 1.4–2.3x vs Triton 基线; 1.1–1.3x vs ALTER (bf16 prefill)

形状最快

1.7–2.1x

MoE 小 batch (warp-decode) : vs Triton; 1.1–1.6x vs ALTER; 中 batch 保持 0.9x

1.6–3.6x

GPT-OSS 120B 端到端输出吞吐：Gluon 路径 vs 可移植 Triton 路径 (20 个测量点全部更优)

同一份 Runtime、同一个公开 API —— AMD 性能来自『注册了更好的 kernel』，而非另写一套 serving 路径

来源: pytorch.org/blog/lightseek-tokenspeed-kernel/ (GPT-OSS 120B on MI355X, ROCm 7.2.1, EvalScope 评测)

05

PART 05

工程实践与性能收益

用数据说话：Runtime 调度、MLA Kernel、推测解码与端到端收益

Runtime 调度效率

MLA Kernel 效果

Spec Decoding

端到端性能

Runtime 调度效率提升 (vs TensorRT-LLM)

对比方法: TensorRT-LLM (当前 NVIDIA 上 SoTA) · Blackwell · 单机非 PD · Attention TP4 + MoE TP4 · SWE-smith agentic 流量 · EvalScope 评测

TokenSpeed 在整个 Pareto 前沿上不劣于 TRT-LLM: 低延迟端更快, 高吞吐端持平或更优

~9%

bs=1 延迟

最小延迟场景快于 TRT-LLM (同 batch 下)

~11%

吞吐

≈100 TPS/user 时每 GPU 吞吐更高

500+

TPS/user

bs=1 四种并行配置 (TP4/TP4EP4/TP8/TP8EP8) 全部达成, TP8 峰值 580

调度效率来自哪里?



低开销 CPU 入口

SMG 集成的 AsyncLLM: 请求处理不占 GPU 关键路径



inter-graph 零等待

pinned 内存非阻塞拷贝 · host 计数器轮询 · 层级 event barrier



图化 decode

target + sampler + draft 单 CUDA graph 一次 launch

来源: lightseek.org/blog/lightseek-tokenspeed.html; pytorch.org/blog/up-to-580tps...

MLA Kernel 优化效果 (vs TensorRT-LLM MLA)



MLA Prefill: 5 / 5 场景全面领先

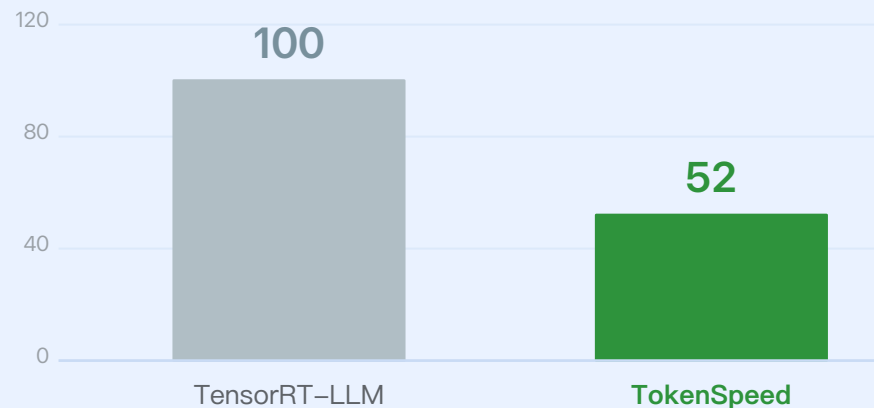
5 种典型 coding-agent prefill 负载（长前缀 KV cache），TokenSpeed MLA prefill 全部优于 TRT-LLM

- ✓ ragged varlen 无 padding
- ✓ 调优 softmax (AOT binary)
- ✓ Q/KV 不同 seq 同包处理
- ✓ PDL + skip-correction

MLA Decode: 延迟近乎减半

bs = 4 / 8 / 16, 长前缀 KV cache + 推测解码场景, decode 延迟相对 TRT-LLM 近乎减半

MLA decode 延迟 (归一化, 越低越好)



bs 4/8/16 · 长前缀 KV · 推测解码场景 (源报告表述为『近乎减半』)

Prefill 5 场景对比 (延迟越低越好)

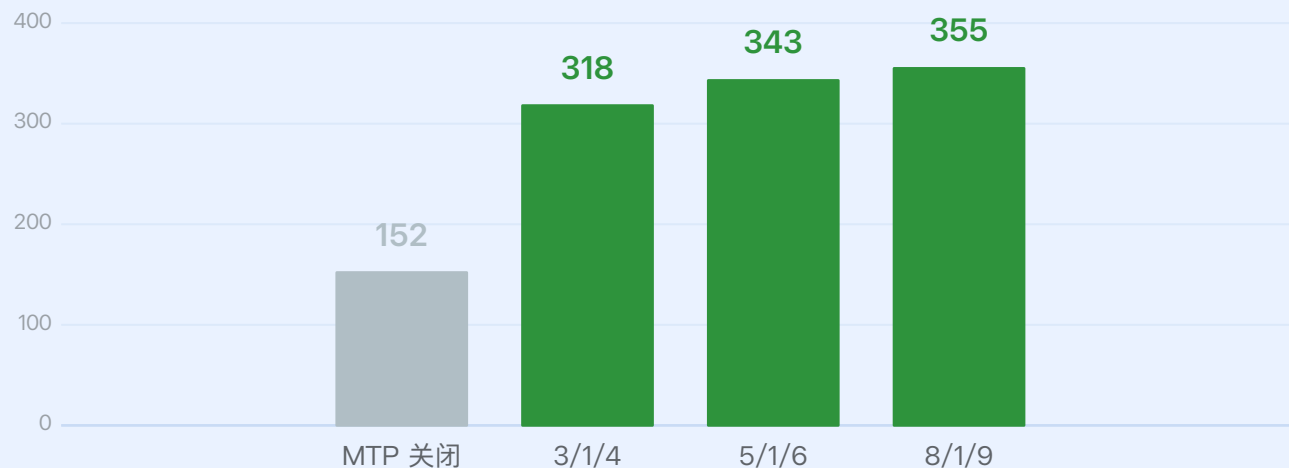
q=1K / kv=32K	✓ 领先
q=4K / kv=32K	✓ 领先
q=1K / kv=64K	✓ 领先
q=4K / kv=64K	✓ 领先
q=8K / kv=64K	✓ 领先

Speculative Decoding 加速：Inkling 975B 实测

场景：Inkling 975B（每 token 激活 41B）· 4x NVIDIA B200 · NVFP4 · agentic 多轮负载（50K+ 上下文 · 10–15 轮 · ~90% 前缀命中）

单用户解码速度（tokens/s），MTP 步数配置对比 —— 每次迭代平均推进 3.3 token

MTP 步数 vs 单用户解码速度



2.33×

MTP 8 步相对关闭（354.6 vs 152.4 tps/user）

2.4×

同一配置迁移到 AMD MI355X（MXFP4，4 卡）单用户解码加速

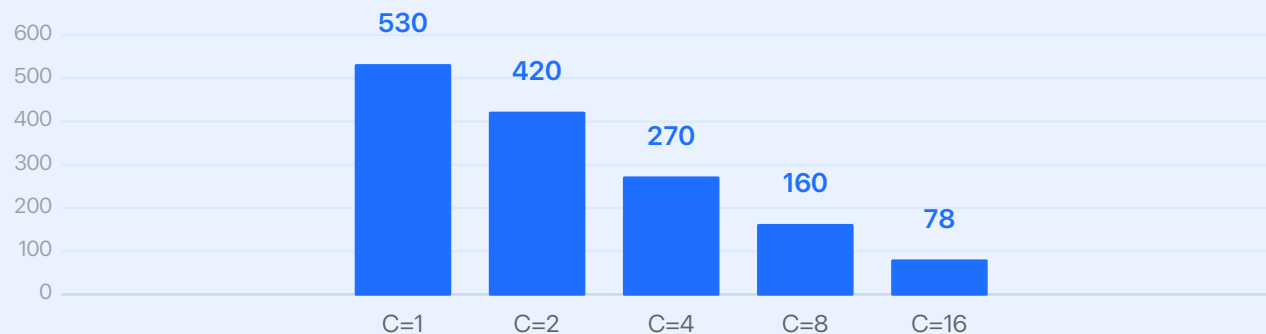
端到端：Qwen3.5-397B-A17B 创纪录 580 TPS

580
TPS / user

Qwen3.5-397B-A17B · NVFP4 · B200×8 · TP8 ·
bs=1 · agentic 负载

四种并行配置 (TP4/TP4EP4/TP8/TP8EP8) bs=1 全部
500+

并发扩展：单用户延迟 vs 并发数 (agentic 负载)



吞吐与延迟可调

纯 TP 与 TP+EP 同卡数下接近 —— 部署灵活



90%+ 前缀命中

多轮 agentic 负载下 prefill 开销大幅降低

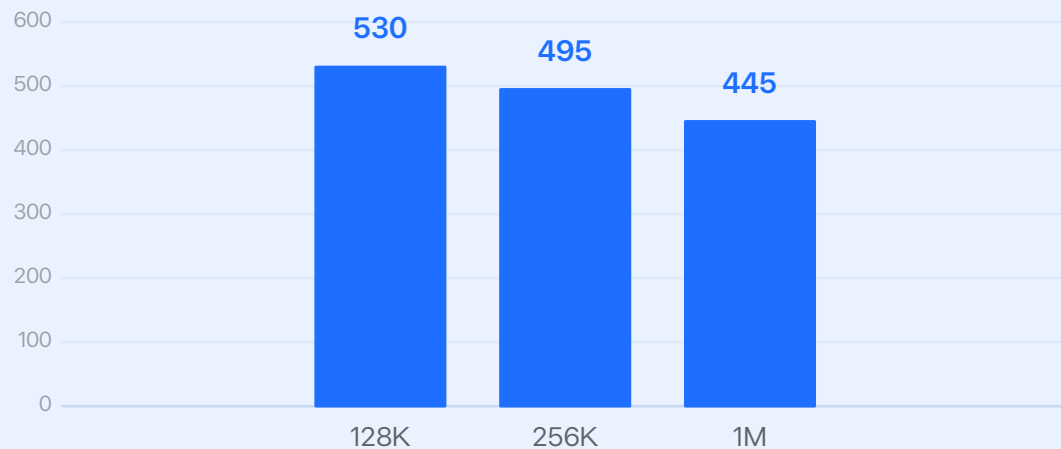


MTP 收益显著

bs=1 时 +100%~+159%；长输出 bs=32/64 仍 +38%~+90%

跨模型验证与长上下文能力

Qwen3.5-397B 长上下文解码吞吐 (NIAH 1M 切片)



128K → 1M 端到端解码吞吐衰减仅 ~16% (单位: TPS/user)



Kimi K2.5

SWE-Smith agentic · NVFP4: conc=1 时 423 TPS/user



Kimi K3

Day-0: 发布一周内完成支持; AMD 上 KDA decode 16K 上下文 134.78x vs Triton



Inkling

975B · MXFP4 4xMI355X 可部署; MTP 单用户解码 2.4x; 质量与 BF16 相当



GPT-OSS 120B

MI355X 单卡: Gluon 端到端 1.6-3.6x vs Triton; kernel 被 vLLM 采用

跨模型 = 跨架构验证: MoE / MLA / 混合线性注意力 / 视觉 (K3 MoonViT) / 4-bit 量化 —— 同一引擎全部覆盖

统一 Kernel API + 统一缓存语义 (FlatKV / HybridPrefixCache / COW) 是『新模型 Day-0 支持』的基础设施

来源: pytorch.org/blog (580 TPS 长上下文评测); lightseek.org/blog (K3 / Inkling); 仓库 [test/ci/perf](https://github.com/pytorch/test/ci/perf)

总结：把系统当作系统来优化

五条核心设计原则

01

目标先变

Agentic 下优化的是『延迟约束下的吞吐』—— 用 Pareto 前沿而不是单一指标衡量引擎

02

分层清晰

Control (C++ 安全) / Execution (Python 效率) / Kernel (可插拔专业)，每层独立演进

03

并行交给编译

Placement 注解 → SPMD 编译器自动生成通信与融合 (Deferred Reduce)

04

Kernel 与模型解耦

Registry + Selector: 新硬件 = 注册新 kernel; 可移植性与性能并存

05

数据说话

580 TPS · 134.78x · 2.33x —— 每层优化都进入 CI 性能门禁，可复现

TokenSpeed 已开源 (MIT) · github.com/lightseekorg/tokenspeed · 欢迎共建与性能 PR

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



Q & A

谢谢聆听 · 欢迎交流



github.com/lightseekorg/tokenspeed

开源仓库 (MIT)



lightseek.org

博客 / 发布文章



pytorch.org/blog

TokenSpeed-Kernel 与 580 TPS 深度解读

感谢生态伙伴: NVIDIA DevTech · AMD · Qwen 团队 · Moonshot AI · Thinking Machines Lab · Mooncake · EvalScope · FluentLLM